

Constraint-Based Problem Solving

Report — Constraint Analysis for Questions 1, 2 and 3

Question 1: Hospital Shift Scheduling — Constraint Analysis

This is modelled as a Constraint Satisfaction Problem (CSP):

- Variables: D1, D2, D3
- Domain of each variable (before pruning): {Morning, Afternoon, Night}
- Constraints: unary constraints (i) $D1 \neq \text{Night}$ and (iv) $D3 \neq \text{Morning}$ prune the initial domains before search even begins; binary constraint (iii) all-different enforces one doctor per shift; ordering constraint (ii) $D2 < D3$ restricts the relative shift order; constraint (v) is satisfied automatically since every variable is assigned a value from the domain.

Applying unary constraints first (a standard CSP pre-processing step) reduces the search space immediately:

- $D1 \in \{\text{Morning, Afternoon}\}$ (Night eliminated by constraint i)
- $D2 \in \{\text{Morning, Afternoon, Night}\}$ (no unary restriction)
- $D3 \in \{\text{Afternoon, Night}\}$ (Morning eliminated by constraint iv)

Variable ordering used for the search: $D1 \rightarrow D2 \rightarrow D3$. Value ordering within each domain: Morning $\rightarrow$ Afternoon $\rightarrow$ Night. This pruning and ordering strategy directly determines how efficiently the backtracking search converges on a valid solution.

Question 2: Robot Grid Navigation — Environment & Constraint Analysis

Grid representation (5 rows $\times$ 5 columns, indexed (row, col) from (0,0) at top-left to (4,4) at bottom-right). 'S' = start, 'G' = goal, 'X' = obstacle, '.' = free cell:

C0	C1	C2	C3	C4
S	.	.	.	.
.	X	.	X	.
.	X	.	.	.
.	X	.	X	.
.	.	.	.	G

Start $S = (0,0)$, Goal $G = (4,4)$. Obstacles = $\{(1,1), (1,3), (2,1), (3,1), (3,3)\}$.

Why BFS is appropriate: since every move has an identical, uniform cost of 1, BFS (which expands nodes level-by-level using a FIFO queue) is guaranteed to find the shortest path in terms of number of moves — this is equivalent to the optimal-cost path here, so BFS behaves as an optimal uninformed search for this problem. Manhattan Distance is used only as an independent check: the straight-line grid distance from $S(0,0)$ to $G(4,4)$ is $|4-0| + |4-0| = 8$, which gives a lower bound on the true path cost. If BFS returns a path of cost exactly 8, the path is provably optimal.

Node expansion order at each cell follows a fixed neighbour priority: Right, Down, Left, Up (R, D, L, U). A cell is skipped if it is out of the 5 $\times$ 5 grid bounds, is an obstacle, or has already been visited.

Question 3: Autonomous Rescue Robot — Constraint Analysis

a) Analysis of Constraints and Their Effect on Decision-Making:

- Movement restricted to 4 directions: reduces the branching factor at every node to at most 4, simplifying the successor function but also meaning the robot cannot take shortcuts through walls or diagonals — it must plan around obstacles explicitly.
- Blocked cells (obstacles): these dynamically prune the state space; because they are not all known upfront, the robot must treat any yet-unseen cell as 'unknown' rather than 'free', and can only confirm it is safe once observed.
- Limited sensing (adjacent cells only): this is what makes the environment partially observable to the robot. The robot cannot commit to a full global plan in advance; it must interleave sensing and acting, replanning each time new information is revealed.
- Variable move cost (1 normally, 3 in a risky zone): this turns the problem from a simple shortest-path (unit cost) search into a weighted-cost search, so an algorithm that only counts steps (like plain BFS) is no longer guaranteed optimal — path cost, not path length, must now be minimized.
- Minimizing total cost under safety constraints: creates a trade-off — the shortest path in terms of steps may pass through risky zones and be more expensive, while a longer path avoiding risky zones may have lower total cost. The search must compare cumulative path cost, not hop count.
- Dynamic knowledge update (online search): because the robot only learns the map as it senses it, it cannot run a one-shot offline search; it must repeatedly search-act-sense-update in a loop, correcting its plan whenever a previously assumed-free cell turns out to be blocked, or a risky zone is discovered.